Python——Selenium快速上手+方法

一、Selenium是什么

Selenium是一个用于自动化web应用程序测试的工具。它提供了一组工具和库，可以用多种编程语言（如Java、Python、C#等）编写测试脚本，模拟用户在浏览器中的行为，如点击链接、填写表单、提交数据等。Selenium可以在各种浏览器上运行，包括Chrome、Firefox、Safari等，它还可以与其他测试框架和工具集成，帮助开发人员和测试人员自动化执行各种测试任务，提高测试效率和质量。

二、Python安装Selenium

1、安装Selenium第三方库

4.6低版本的为例

```
# 换源
pip config set global.index-url https://pypi.tuna.tsinghua.edu.cn/simple

# 安装我这个版本
pip install selenium==4.6.0

# 安装默认最新版本
pip install selenium
```

下载Selenium后，我们需要他来打开浏览器，

但是Selenium需要有浏览器驱动，才可以打开浏览器

2、下载浏览器驱动

下面是Selenium目前支持的浏览器

Chrome:	https://chromedriver.chromium.org/downloads
Edge:	https://developer.microsoft.com/en-us/microsoft-edge/tools/webdriver/
Firefox:	https://github.com/mozilla/geckodriver/releases
Safari:	https://webkit.org/blog/6900/webdriver-support-in-safari-10/

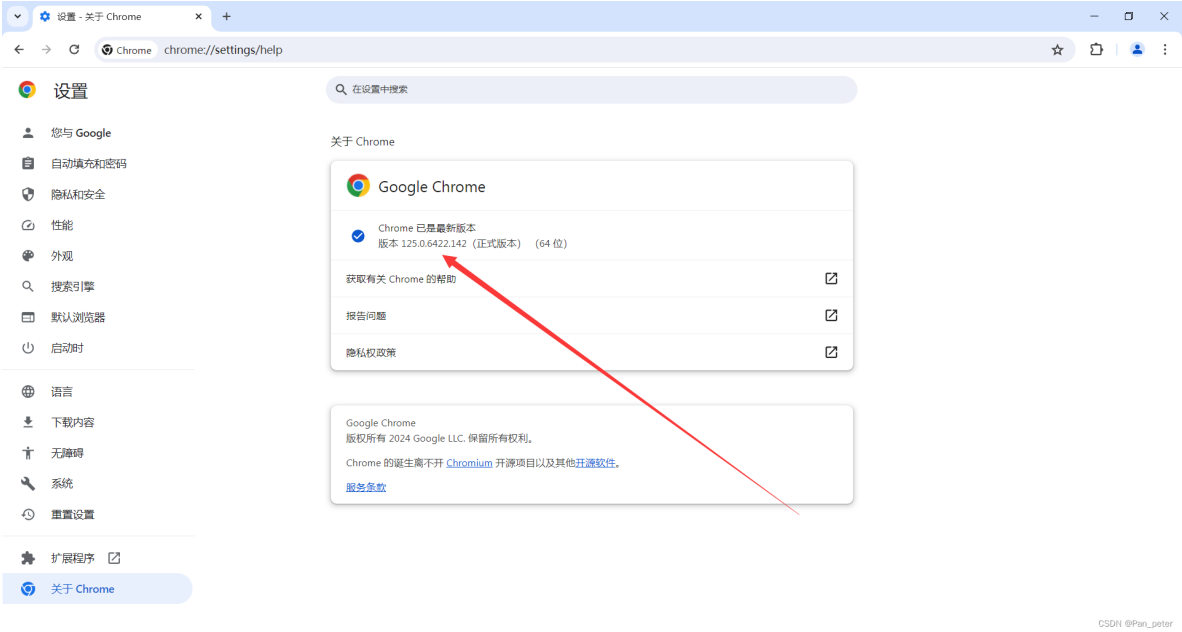
CSDN @Pan_peter

先确定自己的谷歌浏览器的版本

打开谷歌浏览器，输入地址：

```
chrome://settings/help
```

可以查看自己的版本：



选择对应版本号的驱动版本

下载地址：
https://chromedriver.storage.googleapis.com/index.html

这里记录下最新版本谷歌驱动器下载地址：
https://googlechromelabs.github.io/chrome-for-testing/

3、使用Python来打开浏览器

一般情况下，存放在python的安装路径，方便了我们自己使用，可以快速跑代码demo

但是，当需要项目打包，在其他电脑上运行的时候，浏览器驱动存放位置需要放在合适位置

这里直接放在D盘作为演示

```
from selenium import webdriver

# 设置Chrome浏览器驱动程序的路径
chrome_driver_path = "D:\\chromedriver.exe"

# 初始化Chrome浏览器
browser = webdriver.Chrome(executable_path=chrome_driver_path)

# 打开百度网页
browser.get("https://www.baidu.com")

# 这里可以编写其他与页面交互的代码

# 关闭浏览器
browser.quit()
```

运行结果：
代码可能无法成功运行，因为未正确配置ChromeDriver的路径或版本不匹配。此外，Selenium的旧版本使用executable_path参数，新版本已改为service对象方式配置，可能导致运行失败。

当我们看见百度网页后，浏览器就会关闭，这样就说明我们设置完成了

三、Selenium的初始化

当我们创建浏览器驱动对象的时候，他会**创建一个新的干净的浏览器**，来供我们使用

```
from selenium import webdriver

# 设置Chrome浏览器驱动程序的路径
chrome_driver_path = "D:\\chromedriver.exe"

# 初始化Chrome浏览器
browser = webdriver.Chrome(executable_path=chrome_driver_path)
```

所以，我们在浏览器上设置的东西，也可以通过代码实现设置好

比如：

- 浏览器下载文件后，下载的地址设置
- 浏览器是否加载图片
- 浏览器是否禁用JS
- 浏览器是否使用隐私模式
- 浏览器是否使用缓存
- .....

因此，我们可以实现封装一个类，来实现浏览器的初始化

编写简单类，可以通过json文件来修改参数

比如：

获取驱动的路径（在打包后，我们的驱动也需要在生产环境中，让程序可以找到，这时候我们灵活修改）

是否加载图片

是否使用缓存

.....

```
# -*- coding: utf-8 -*-
# @Author : pan
# @Description : 获取浏览器驱动模块
# @Date : 2024年5月25日17:53:17
import time
import json
import os
import shutil  # 提前导入，避免延迟导入
from dataclasses import dataclass, asdict
from selenium import webdriver
from selenium.webdriver.chrome.service import Service  # Selenium 4.x 新增Service
类
from selenium.common.exceptions import WebDriverException  # 异常捕获

@dataclass
class WebDriveConf:
    """浏览器驱动配置（优化：布尔值更直观）"""
    chromedriver_url: str = 'D:\\chromedriver.exe'
    user_data_dir: str = 'D:\\webDriverTempDir'
    is_use_cache: bool = False   # False=禁用缓存，True=启用
```

```
    is_load_page: bool = True    # True=有界面，False=无界面
    is_load_img: bool = False    # False=禁用图片，True=启用
    is_ignore_certificate: bool = True  # True=忽略证书
    is_incognito: bool = True    # True=启用无痕
    page_load_timeout: int = 30  # 新增：页面加载超时时间（秒）

    def to_dict(self) -> dict:
        return asdict(self)

    def read_json_config(self, file_path: str) -> dict:
        if not os.path.exists(file_path):
            with open(file_path, "w", encoding="utf-8") as file:  # 新增：指定编码
                json.dump(self.to_dict(), file, indent=4, ensure_ascii=False)
        with open(file_path, "r", encoding="utf-8") as file:  # 新增：指定编码
            return json.load(file)

    def write_json_config(self, config_dict: dict, file_path: str):
        with open(file_path, "w", encoding="utf-8") as file:  # 新增：指定编码
            json.dump(config_dict, file, indent=4, ensure_ascii=False)

class ChromeWebDriver:
    def __init__(self, conf: WebDriveConf):
        self.conf = conf
        self.chrome_opt = webdriver.ChromeOptions()
        self.chromedriver = None  # 初始化驱动为None

        # 1. 配置用户数据目录
        self.chrome_opt.add_argument(f'--user-data-dir=
{self.conf.user_data_dir}')

        # 2. 清理缓存
        if not self.conf.is_use_cache and
os.path.exists(self.conf.user_data_dir):
            try:
                shutil.rmtree(self.conf.user_data_dir)
                print(f"已清理缓存目录：{self.conf.user_data_dir}")
            except Exception as e:
                print(f"清理缓存失败：{e}")

        # 3. 创建临时目录
        if not os.path.exists(self.conf.user_data_dir):
            try:
                os.makedirs(self.conf.user_data_dir)
                print(f"已创建临时目录：{self.conf.user_data_dir}")
            except Exception as e:
                print(f"创建临时目录失败：{e}")

        # 4. 配置Chrome参数
        if not self.conf.is_load_page:
            self.chrome_opt.add_argument('--headless=new')  # 新版headless参数
（Selenium 4.10+）
            self.chrome_opt.add_argument('--disable-gpu')  # 配合headless

        if not self.conf.is_load_img:
            self.chrome_opt.add_argument('--blink-settings=imagesEnabled=false')

        if self.conf.is_ignore_certificate:
            self.chrome_opt.add_argument('--ignore-certificate-errors')
```

```
            self.chrome_opt.add_argument('--ignore-ssl-errors')  # 新增：忽略SSL错误

        if self.conf.is_incognito:
            self.chrome_opt.add_argument('--incognito')

        # 5. 创建驱动实例（兼容Selenium 4.x）
        try:
            service = Service(executable_path=self.conf.chromedriver_url)
            self.chromedriver = webdriver.Chrome(service=service,
options=self.chrome_opt)
            # 设置页面加载超时
            self.chromedriver.set_page_load_timeout(self.conf.page_load_timeout)
            print("Chrome驱动初始化成功")
        except WebDriverException as e:
            print(f"驱动初始化失败：{e}")
            raise  # 抛出异常，终止程序

    def __del__(self):
        """析构函数：确保浏览器进程关闭"""
        if self.chromedriver:
            try:
                self.chromedriver.quit()
                print("浏览器已关闭")
            except Exception as e:
                print(f"关闭浏览器失败：{e}")

if __name__ == '__main__':
    try:
        # 读取配置并初始化
        conf_dict = WebDriveConf().read_json_config("web_driver_conf.json")
        web_driver_conf = WebDriveConf(**conf_dict)
        chrome_web_driver = ChromeWebDriver(web_driver_conf)

        # 打开网页
        if chrome_web_driver.chromedriver:
            chrome_web_driver.chromedriver.get("https://www.bilibili.com/")
            time.sleep(10)
    except Exception as e:
        print(f"执行失败：{e}")
    finally:
        # 确保浏览器关闭
        if 'chrome_web_driver' in locals() and chrome_web_driver.chromedriver:
            chrome_web_driver.chromedriver.quit()
```

四、Selenium获取网页元素

当我们打开了网页，我们就可以获取网页上的**任何东西**

当然，一般我们只需要获取网页上的一些数据信息，文本内容

比如：获取元素中的文本内容

```python
# 通过XPath选择元素
element = driver.find_element_by_xpath("//h1")

# 获取元素中的文本
text = element.text

print("元素中的文本是:", text)
```

获取元素的属性内容

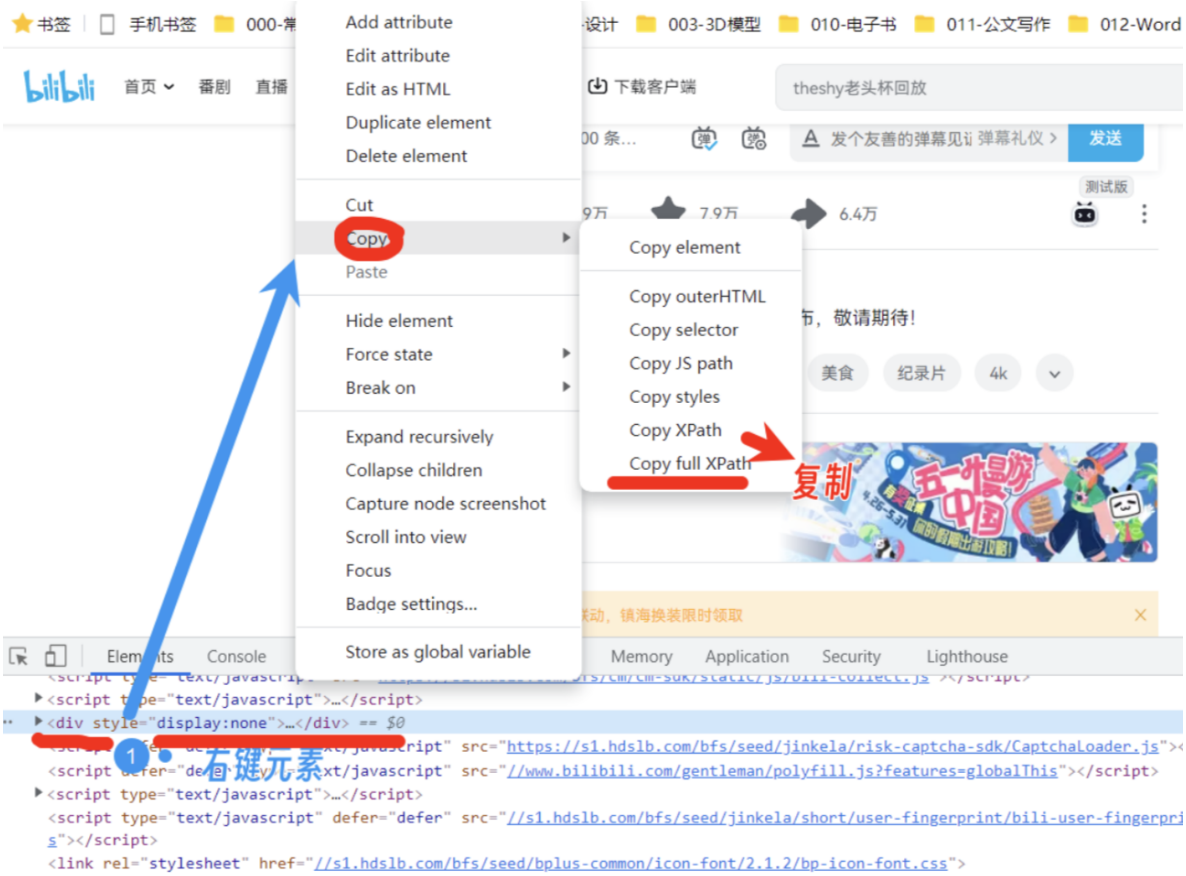
```python
# 通过XPath选择元素
element = driver.find_element_by_xpath("//h1")

# 获取元素中的class内容
class_content = element.get_attribute("class")

print("元素中的class内容是:", class_content)
```

Selenium有很多元素筛选方法，通过class、id等等进行筛选

方法	描述	参数	示例
findElement(By.id())	通过元素的 id 属性值来定位元素	对应的id属性值	findElement(By.id("kw"))
findElement(By.name())	通过元素的 name 属性值来定位元素	对应的name值	findElement(By.name("user"))
findElement(By.className())	通过元素的 class 名来定位元素	对应的class类名	findElement(By.className("passworld"))
findElement(By.tagName())	通过元素的 tag 标签名来定位元素	对应的标签名	findElement(By.tagName("input"))
findElement(By.linkText())	通过元素🔍标签对之间的文本信息来定位元素	文本内容	findElement(By.linkText("登录"))
findElement(By.partialLinkText())	通过元素🔍标签对之间的部分文本信息来定位元素	部分文本内容	findElement(By.partialLinkText("百度"))
findElement(By.xpath())	通过xpath语法来定位元素	xpath表达式	findElement(By.xpath("//input[@id='kw']"))
findElement(By.cssSelector())	通过css选择器来定位元素	css元素选择器	findElement(By.cssSelector("#kw"))

CSDN @Pan_peter

xpath可以直接下图方法使用这个来获取这个元素的路径



4.1 获取元素的实用方法

打开网页后，需与页面元素（如按钮、输入框）交互，因此要先定位并获取元素。

1. 模糊匹配 + 多条件联合定位（XPath）

通过 XPath 同时筛选元素的**多个属性 / 文本**，精准定位目标元素：

```
# 示例：定位class为"fancytree-title"且title包含"干线"的span标签
main_line_list = chromedriver.find_elements(
    By.XPATH,
    "//span[@class='fancytree-title' and contains(@title, '干线')]"
)
```

- 逻辑：同时筛选 `class` 属性和 `title` 属性的模糊匹配（`contains` 表示 “包含”）；
- 优势：XPath 支持**模糊匹配（contains）、多条件联合（and）**，比单一属性定位更灵活，是 Selenium 中推荐的定位方式。

2. 结合显式等待 + Lambda 表达式定位

用 `WebDriverWait`（显式等待）配合 Lambda 表达式，更直观地表达定位逻辑，同时避免 “元素未加载完成” 的报错：

```
# 示例：等待10秒，直到ID为"tree-input"的元素加载完成
element = WebDriverWait(chromedriver, timeout=10).until(
    lambda d: d.find_element(By.ID, "tree-input")
)
```

- 逻辑：`WebDriverWait` 会循环执行 Lambda 中的定位操作，直到元素出现（或超时）；
- 优势：代码更简洁，同时解决了 “页面加载慢导致元素定位失败” 的问题。

3.增加等待时间，提升稳定性

页面操作（如点击、跳转）后需等待页面加载，避免因元素未渲染完成导致的操作失败：

```
# 示例：点击“重置密码”后等待2秒
chrome_web_driver.chromedriver.find_element(
    By.XPATH, "//span[text()='重置密码']"
).click()
time.sleep(2)  # 强制等待2秒
```

- 说明：`time.sleep()`是**强制等待**（简单但不够灵活）；实际场景更推荐用`WebDriverWait`（显式等待），仅在元素加载完成后执行操作。

4. 利用树形结构（层级定位）

当元素存在重复属性时，先定位**父元素**，再在父元素范围内定位目标子元素：

```
# 示例：先定位ID为"cold-19438634437413"的父元素，再定位其下class为"suffix-group"的子元素
chromedriver.find_element(
    By.ID, "cold-19438634437413"
).find_element(By.CLASS_NAME, "suffix-group")
```

- 优势：缩小定位范围，避免定位到其他区域的重复元素。

5. 获取多个元素（批量操作）

需批量操作元素时，用`find_elements`（复数形式）获取元素列表，再遍历操作：

场景 1：定位输入框（按顺序填充账号 / 密码）

```
# 1. 获取所有input标签元素
input_elements = chromedriver.find_elements(By.TAG_NAME, 'input')
# 2. 打印input元素数量
print(len(input_elements))
# 3. 第一个input填账号，第二个填密码
input_elements[0].clear()  # 清空输入框
input_elements[0].send_keys(username)  # 输入账号
time.sleep(1)
input_elements[1].clear()
input_elements[1].send_keys(password)  # 输入密码
```

- 说明：需确保页面中`input`元素的顺序固定（通常第一个是账号、第二个是密码）。

场景 2：批量点击多个元素

```
# 1. 获取所有title包含“架空线路”的span元素
overhead_line = chromedriver.find_elements(
    By.XPATH, "//span[@class='fancytree-title' and @title='架空线路']"
)
time.sleep(3)
# 2. 遍历元素列表，逐个点击
for line in overhead_line:
    line.click()
```

- 注意：批量点击前需确保元素已完全加载，避免操作失败。

补充说明

1. 定位方式优先级：推荐：XPath（灵活）>ID（唯一）>CLASS_NAME（易重复）>TAG_NAME（范围大）；
2. 显式等待替代强制等待：**time.sleep()**是“一刀切”的等待，实际测试中更推荐用WebDriverWait，例如：

```
WebDriverWait(chromedriver, 10).until(
    EC.element_to_be_clickable((By.XPATH, "//span[text()='重置密码']"))
).click()
```

3. 元素定位失败的常见原因：
    - 元素在`iframe`内（需先切换`frame`）；
    - 元素是动态加载的（需等待）；
    - 定位表达式错误（需检查 XPath/ID 是否正确）。

4.2 获取网页元素失败

当 Selenium 定位元素失败时，可按以下步骤排查解决：

1. 确认定位方法是否正确

- 首先确保已正确使用 4.1 中提到的定位方法（如 XPath 多条件匹配、层级定位等）；
- 若用`find_elements`（复数）获取到多个元素，需确认是否定位到了**目标元素**（可通过索引筛选，或优化定位表达式）。

2. 检查代码中的元素标签 / 属性是否正确

- 需明确目标元素的标签类型（是`<div>`、`<span>`还是`<a>`等），标签错误是常见定位失败原因（例如把`<span>`写成`<div>`）；
- 同时检查元素的属性（如`id`、`class`）是否拼写正确。

3. 处理 iframe 嵌套问题

网页中的`<iframe>`（内联框架）是独立的文档，Selenium 默认无法直接定位 iframe 内的元素，需先**切换 iframe 上下文**：

步骤 1：定位 iframe 元素

```
# 定位id为"frame_id"的iframe元素（可根据实际属性调整定位方式）
iframe = driver.find_element(By.XPATH, "//iframe[@id='frame_id']")
```

步骤 2：切换到 iframe 上下文

```
# 切换到目标iframe的上下文，之后才能定位iframe内的元素
driver.switch_to.frame(iframe)
```

步骤 3：操作 iframe 内的元素

```
# 定位iframe内id为"button_inside_iframe"的按钮元素
element_inside_iframe = driver.find_element(By.XPATH,
"//button[@id='button_inside_iframe']")
# 点击该按钮
element_inside_iframe.click()
```

步骤 4：切回默认上下文

```
# 操作完成后，切回主页面的上下文（否则无法定位主页面元素）
driver.switch_to.default_content()
```

补充：多层 iframe 的处理

若页面存在多层嵌套的 iframe（如 iframe 内还有 iframe），需**逐层切换**：

```
# 1. 定位外层iframe并切换
outer_iframe = chromedriver.find_elements(By.TAG_NAME, "iframe")[0]
chromedriver.switch_to.frame(outer_iframe)

# 2. 定位内层iframe并切换
inner_iframe = chromedriver.find_elements(By.TAG_NAME, "iframe")[1]
chromedriver.switch_to.frame(inner_iframe)

# 3. 操作内层iframe内的元素...

# 4. 切回外层iframe（如需继续操作外层）
chromedriver.switch_to.parent_frame()
```

补充：其他常见失败原因及解决方法

1.元素未加载完成：

用**显式等待**替代强制等待，确保元素可交互后再操作：

```
from selenium.webdriver.support import expected_conditions as EC
from selenium.webdriver.support.ui import WebDriverWait

# 等待10秒，直到元素可点击
WebDriverWait(driver, 10).until(
    EC.element_to_be_clickable((By.ID, "target_btn"))
).click()
```

2.元素被遮挡 / 不可见

检查是否有弹窗、遮罩层遮挡元素，可先关闭遮挡物，或通过`execute_script`强制点击：

```
element = driver.find_element(By.ID, "target_btn")
# 用JS强制点击（绕过可见性检查）
driver.execute_script("arguments[0].click();", element)
```

3.动态元素（属性 / 文本变化）：避免用动态变化的属性（如随机生成的`id`）定位，改用相对稳定的属性（如`class`、`text()`）或层级定位。

4.3 Selenium 网页窗口切换解决方案

网页跳转的问题

当操作网页触发新窗口打开时，Selenium 默认仍停留在原窗口上下文，若需操作新窗口元素，必须**切换到新窗口的句柄**。

核心概念：窗口句柄（Window Handle）

每个浏览器窗口对应一个唯一的“句柄”（字符串标识），Selenium 通过句柄区分不同窗口。

步骤 1：获取当前窗口句柄

```
# 获取当前（原）窗口的句柄，用于后续区分新窗口
current_window_handle = driver.current_window_handle
```

步骤 2：获取所有窗口句柄

```
# 获取浏览器当前所有打开窗口的句柄列表（包含原窗口+新窗口）
all_window_handles = driver.window_handles
```

步骤 3：切换到新窗口句柄

```
# 遍历所有窗口句柄
for window_handle in all_window_handles:
    # 判断当前句柄是否不是原窗口句柄（即新窗口）
    if window_handle != current_window_handle:
        # 切换到新窗口的上下文
        driver.switch_to.window(window_handle)
        # 找到新窗口后退出循环（避免遍历后续无关窗口）
        break
```

步骤 4：切换回原始窗口句柄（可选）

```
# 操作完新窗口后，切回原窗口的上下文
driver.switch_to.window(current_window_handle)
```

补充说明与优化

多新窗口的处理：若打开了多个新窗口，可通过窗口标题 / URL 辅助区分（避免仅依赖句柄匹配）：

```
for window_handle in all_window_handles:
    # 切换到当前遍历的窗口
    driver.switch_to.window(window_handle)
    # 判断窗口标题是否包含目标关键词（如“新页面”）
    if "新页面" in driver.title:
        break  # 找到目标窗口后停止
```

窗口切换后的验证：切换后建议验证窗口状态（避免切换失败）：

```
# 切换后打印新窗口的标题，确认是否正确
print("当前窗口标题：", driver.title)
```

旧版语法兼容：Selenium 4.0 + 也支持`driver.switch_to.window()`，与旧版语法一致（无兼容性问题）

常见问题：

- 切换前未等待新窗口加载：需用`WebDriverWait`等待窗口数量变化：

```
from selenium.webdriver.support.ui import WebDriverWait
# 等待窗口数量变为2（原窗口+新窗口）
WebDriverWait(driver, 10).until(lambda d: len(d.window_handles) == 2)
```

五、Selenium点击元素的方法

获取页面元素后，常见操作是**输入内容**和**点击元素**，以下是具体实现方式：

1. 输入内容（文本框操作）

输入前建议先清空内容，避免旧数据干扰

```
# 1. 定位元素：先找到ID为"findSource"的父元素，再定位其下的input标签
input_element = chromedriver.find_element(
    By.ID, 'findSource'
).find_element(By.TAG_NAME, 'input')

# 2. 清空输入框（避免原有内容影响输入结果）
input_element.clear()

# 3. 输入内容（示例：输入“无人机”）
input_element.send_keys("无人机")
```

2. 右键点击元素

需借助`ActionChains`类模拟鼠标右键操作：

```
# 1. 导入ActionChains类（用于模拟鼠标/键盘操作）
from selenium.webdriver.common.action_chains import ActionChains

# 2. 定位目标元素（示例：定位canvas标签元素）
canvas_element = chromedriver.find_element(By.TAG_NAME, 'canvas')

# 3. 初始化ActionChains，绑定驱动对象
action = ActionChains(chromedriver)
# 4. 模拟右键点击元素，最后调用perform()执行操作
action.context_click(canvas_element).perform()
```

3. 左键点击的多种方法

以下是常见的左键点击实现方式，适用于不同场景：

方法	描述 & 代码示例
click () 方法（最常用）	Selenium 原生点击，直接触发元素的点击事件 `python element = driver.find_element(By.ID, "btn") element.click() `
JavaScript 执行点击	绕过元素可见性 / 可点击性限制，强制触发点击（适用于元素被遮挡的场景） `python driver.execute_script("arguments[0].click();", element) `
ActionChains 类	模拟用户真实操作（支持点击、拖拽等复杂动作） `python ActionChains(driver).click(element).perform() `
send_keys(Keys.RETURN)	对输入框类元素，模拟按下回车键触发操作（如搜索框输入后回车搜索） `python from selenium.webdriver.common.keys import Keys input_element.send_keys(Keys.RETURN) `
submit () 方法	仅适用于表单内的提交按钮，提交整个表单 `python form_element = driver.find_element(By.ID, "form") form_element.submit() `
修改元素属性后点击	修复元素的 `onclick` 等属性后再点击（适用于属性异常的元素） `python # 先将元素onclick属性设为null（修复异常） driver.execute_script("arguments[0].onclick = null;", element) element.click() `
pyautogui/Robot 类模拟点击	模拟系统级鼠标点击（适用于极特殊场景，需提前定位元素坐标） `python import pyautogui # 获取元素坐标 x = element.location['x'] y = element.location['y'] pyautogui.click(x, y) `

补充说明

1. 方法优先级建议：
    - 优先用 `click()`：简单直接，符合 Selenium 原生逻辑；
    - 若 `click()` 失败（如元素不可见）：用**JavaScript 点击**；
    - 复杂交互场景（如拖拽后点击）：用 `ActionChains`。
2. 点击失败的常见原因及解决：

    - 元素不可见 / 被遮挡：用`JavaScript点击`绕过；
    - 元素是动态加载的：用`WebDriverWait`等待元素可点击；
    - 元素在 iframe 内：先切换 iframe 上下文再点击；
3. 输入操作的注意事项：
    - 仅`<input>`、`<textarea>`等可输入元素支持`send_keys()`；
    - 输入特殊字符（如回车、空格）需导入`Keys`类（`from selenium.webdriver.common.keys import Keys`）。

在 Selenium 中触发点击事件（最常用的三种方法）

1. 使用`click()`方法（最基础、常用）

这是最简单直接的方式，通过定位元素后调用`click()`方法触发点击。

```
# 1. 定位目标元素：通过ID定位（Selenium 4.x推荐用By.ID，替代旧版find_element_by_id）
from selenium.webdriver.common.by import By  # 需导入By类
element = driver.find_element(By.ID, "element_id")

# 2. 触发元素的点击事件
element.click()
```

2. 使用 JavaScript 执行点击事件（解决稳定性问题）

当`click()`方法出现不稳定（如元素被遮挡、不可见）时，可通过执行 JavaScript 代码强制触发点击。

```
# 1. 定位目标元素
from selenium.webdriver.common.by import By
element = driver.find_element(By.ID, "element_id")

# 2. 执行JavaScript代码：arguments[0]代表传入的element参数，调用其click()方法
driver.execute_script("arguments[0].click();", element)
```

3. 使用`ActionChains`类（模拟用户真实操作）

`ActionChains`用于模拟用户的鼠标 / 键盘操作，适合复杂交互场景（如点击、拖拽、右键等）。

```
# 1. 导入ActionChains类
from selenium.webdriver.common.action_chains import ActionChains
from selenium.webdriver.common.by import By

# 2. 定位目标元素
element = driver.find_element(By.ID, "element_id")

# 3. 初始化ActionChains（绑定driver），调用click()方法，最后用perform()执行操作
ActionChains(driver).click(element).perform()
```

`ActionChains`**的扩展用法**：除了普通点击，还支持双击、右键点击等：

```
# 双击元素
ActionChains(driver).double_click(element).perform()
# 右键点击元素
ActionChains(driver).context_click(element).perform()
```

六、Selenium + browsermob-proxy 获取网络请求

为什么要获取网络请求？

部分数据（如渲染到`<canvas>`中的内容）无法通过页面元素直接提取，需通过**抓包获取接口请求的响应数据**，此时可借助`browsermob-proxy`实现 Selenium 的网络请求捕获。

1. 下载 browsermob-proxy 工具

`browsermob-proxy`是一个代理工具，需先下载其客户端（依赖 Java 环境）：

- 下载地址：browsermob-proxy 的 GitHub Releases 页面
- 环境要求：需安装**JDK 1.8 及以上版本**（推荐 Java 8，可直接安装 JDK 1.8）。

2. 安装 Python 模块`browsermob-proxy`

通过`pip`安装 Python 对应的操作模块：

```
# 执行该命令安装browsermob-proxy的Python客户端
pip install browsermob-proxy
```

3. Selenium + browsermob-proxy 的核心使用流程

```
# 导入内置模块time：用于设置等待时间，确保页面/请求加载完成
import time

# 从selenium库导入webdriver：核心模块，用于操作Chrome浏览器
from selenium import webdriver
# 从browsermobproxy库导入Server：用于启动browsermob-proxy代理服务
from browsermobproxy import Server

# -------------------------- 第一步：启动browsermob-proxy代理服务器 ----------------
----------
# 创建Server实例，指定browsermob-proxy的可执行文件路径（Windows系统为.bat文件）
# 路径说明：r前缀表示原生字符串，避免反斜杠转义问题
server = Server(r"D:\browsermob-proxy-2.1.4\bin\browsermob-proxy.bat")
# 启动browsermob-proxy代理服务（依赖Java环境，需确保已安装JDK并配置环境变量）
server.start()
# 创建一个代理实例：后续Chrome浏览器的所有网络请求都会通过该代理转发
proxy = server.create_proxy()

# -------------------------- 第二步：配置Chrome浏览器使用代理 ----------------------
----
# 拼接代理服务器参数：格式为"--proxy-server=代理地址:端口"，proxy.proxy会自动返回代理的IP
和端口
proxy_address = "--proxy-server={0}".format(proxy.proxy)
# 创建ChromeOptions对象：用于配置Chrome浏览器的启动参数
chrome_options = webdriver.ChromeOptions()
# 添加代理参数：让Chrome使用上述创建的browsermob-proxy代理
chrome_options.add_argument(proxy_address)
# 添加忽略证书错误参数：避免代理拦截HTTPS请求时出现SSL证书验证失败的报错
chrome_options.add_argument('--ignore-certificate-errors')

# 修正：Selenium 4.x中已将chrome_options参数改为options，兼容新旧版本
# 创建Chrome浏览器驱动实例，绑定配置好的代理参数
```

```python
driver = webdriver.Chrome(options=chrome_options)

# -------------------------- 第三步：开始捕获网络请求 --------------------------
# 新建HAR（HTTP Archive）文件，命名为"page"：HAR是存储HTTP请求/响应的标准格式
# 调用new_har后，代理开始记录后续所有的网络请求
proxy.new_har("page")

# -------------------------- 第四步：访问目标网页并等待请求完成 ----------------------
----
# 打开百度首页：触发浏览器的网络请求，这些请求会被代理捕获
driver.get("https://baidu.com")
# 强制等待3秒：确保页面所有网络请求（如图片、接口、脚本）都完成加载，避免漏抓请求
time.sleep(3)

# -------------------------- 第五步：解析并打印捕获的网络请求 ----------------------
---
# 获取捕获的所有HAR格式数据：包含所有请求的URL、请求头、响应头、响应内容等
har = proxy.har

# 遍历所有捕获的请求条目（entry对应一个完整的HTTP请求-响应周期）
for entry in har['log']['entries']:
    # 提取当前请求的URL
    request_url = entry['request']['url']
    # 提取当前请求对应的响应数据（包含状态码、响应头、响应内容等）
    response = entry['response']
    # 简化赋值：将响应数据赋值给response_content（原代码此处无实际意义，保留仅为兼容）
    response_content = response
    # 打印请求URL：便于筛选目标接口
    print("Request URL:", request_url)
    # 打印响应内容：完整的响应数据结构（可根据需求提取具体字段，如response['content']
['text']）
    print("Response Content:", response_content)

# 额外等待3秒：便于查看打印的请求/响应内容（测试用，实际场景可删除）
time.sleep(3)

# -------------------------- 第六步：关闭资源，释放内存 --------------------------
# 关闭Chrome浏览器，终止驱动进程（必须执行，否则会残留浏览器进程）
driver.quit()
# 停止browsermob-proxy代理服务（必须执行，否则会残留代理进程）
server.stop()
```